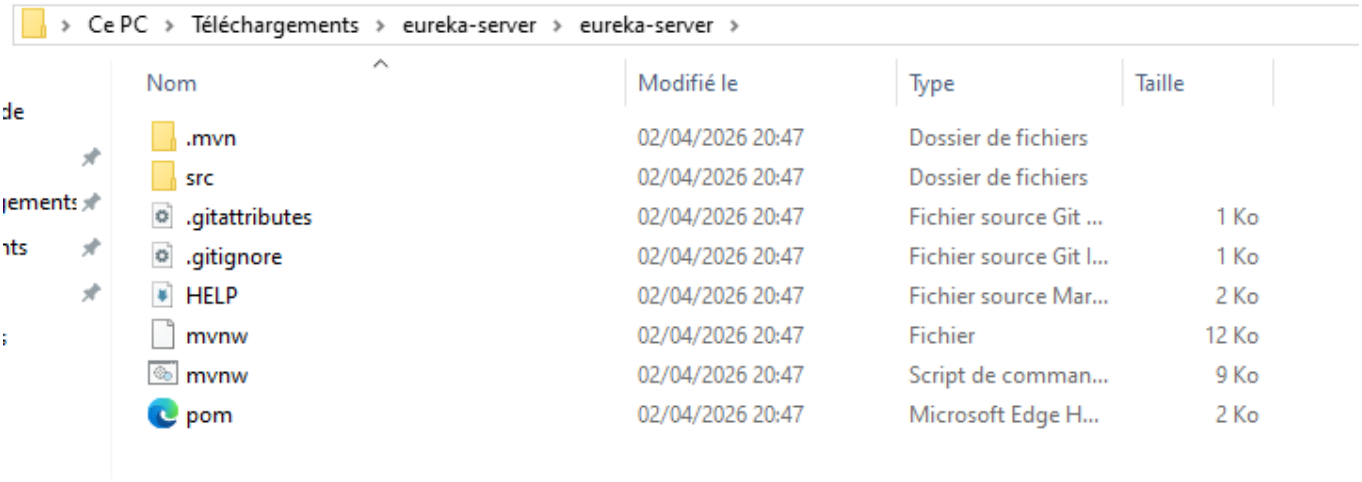
TP°4 : Architecture Orientée Services (SOA)

EL KHAYAR Youssef

Étape 1 : Créer le serveur Eureka:



application.properties | EurekaServerApplication.java

```java
package com.ecommerce.eureka_server;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

localhost:8761

spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-04-02T21:27:34 +0100
Data center	default	Uptime	00:02
		Lease expiration enabled	false
		Renews threshold	1
		Renews (last min)	0

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
No instances available			

General Info

Name	Value
total-avail-memory	156mb
num-of-cpus	4
current-memory-usage	48mb (30%)
server-uptime	00:02
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/,

Étape 2 : Créer le service Product:

start.spring.io

Project
- Gradle - Groovy
- Gradle - Kotlin
- Maven

Language
- Java
- Kotlin
- Groovy

Spring Boot
- 4.1.0 (SNAPSHOT)
- 4.1.0 (M4)
- 4.0.6 (SNAPSHOT)
- 4.0.5
- 3.5.14 (SNAPSHOT)
- 3.5.13

Project Metadata

Group	com.ecommerce
Artifact	product-service
Package name	com.ecommerce. product-service
Packaging	Jar / War
Configuration	Properties / YAML
Java	26 / 25 / 21 / 17

Dependencies — ADD DEPENDENCIES... CTRL + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

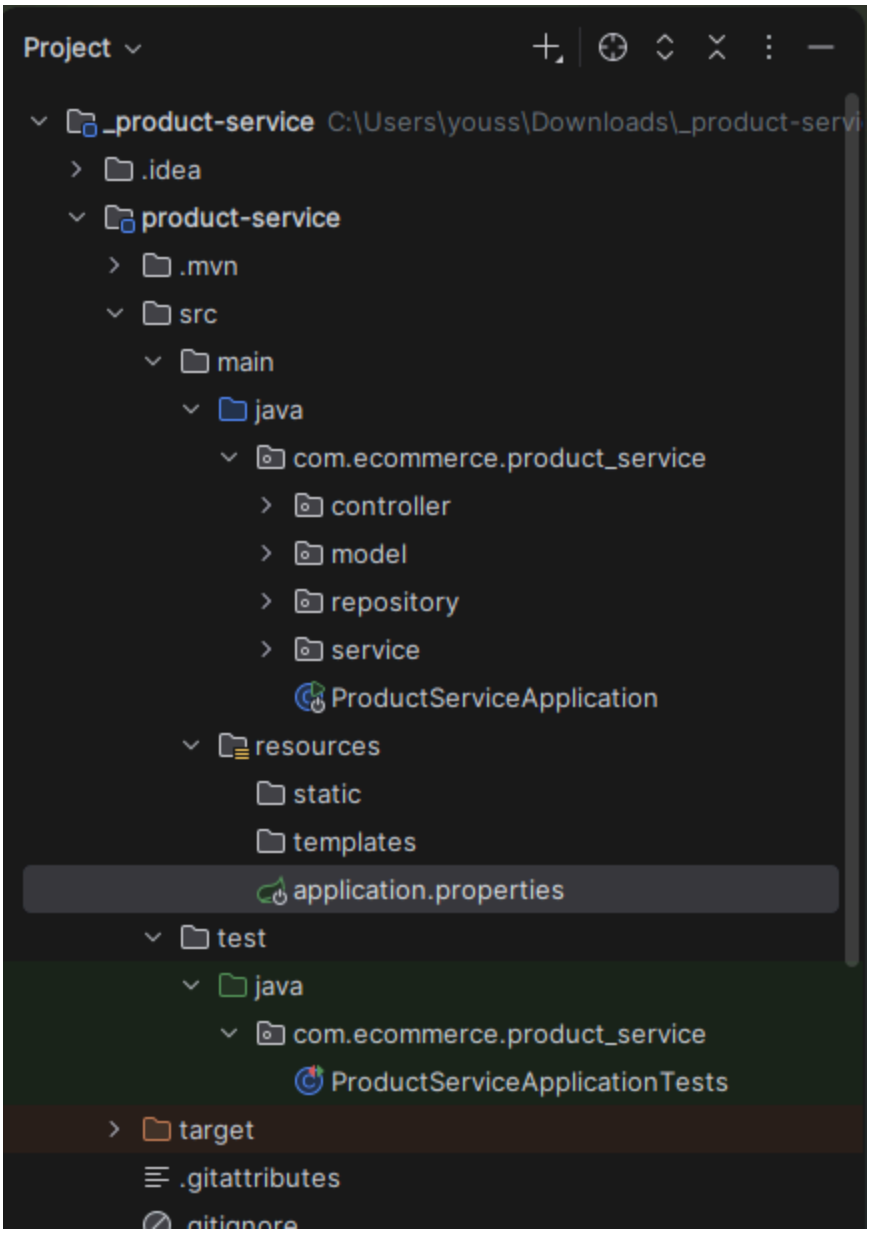
Eureka Discovery Client SPRING CLOUD DISCOVERY
A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

Lombok DEVELOPER TOOLS
Java annotation library which helps to reduce boilerplate code.

GENERATE CTRL + ⏎ | EXPLORE CTRL + SPACE | ...

Ce PC › Téléchargements › _product-service › product-service ›

Nom	Modifié le	Type	Taille
.mvn	02/04/2026 21:34	Dossier de fichiers	
src	02/04/2026 21:34	Dossier de fichiers	
.gitattributes	02/04/2026 21:34	Fichier source Git ...	1 Ko
.gitignore	02/04/2026 21:34	Fichier source Git I...	1 Ko
HELP	02/04/2026 21:34	Fichier source Mar...	2 Ko
mvnw	02/04/2026 21:34	Fichier	12 Ko
mvnw	02/04/2026 21:34	Script de comman...	9 Ko
pom	02/04/2026 21:34	Microsoft Edge H...	4 Ko



application.properties | Product.java | ProductServiceApplication.java | ProductRepository.java

```
server.port=8081
spring.application.name=product-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

```java
package com.ecommerce.product_service.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

```java
package com.ecommerce.product_service.repository;

import com.ecommerce.product_service.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository  no usages
public interface ProductRepository extends JpaRepository<Product, Long> {}
```

ductServiceApplication.java | ProductRepository.java | ProductService.java | ProductController.java

```java
package com.ecommerce.product_service.service;

import com.ecommerce.product_service.model.Product;
import com.ecommerce.product_service.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service  no usages
@RequiredArgsConstructor
public class ProductService {
    private final ProductRepository productRepository;

    public List<Product> getAllProducts() {  no usages
        return productRepository.findAll();
    }

    public Optional<Product> getProductById(Long id) {  no usages
        return productRepository.findById(id);
    }

    public Product createProduct(Product product) {  no usages
        return productRepository.save(product);
    }

    public void deleteProduct(Long id) {  no usages
        productRepository.deleteById(id);
    }
}
```

```java
package com.ecommerce.product_service.controller;

import com.ecommerce.product_service.model.Product;
import com.ecommerce.product_service.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;

    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return productService.getProductById(id) Optional<Product>
                .map(ResponseEntity::ok) Optional<ResponseEntity<...>>
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return productService.createProduct(product);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

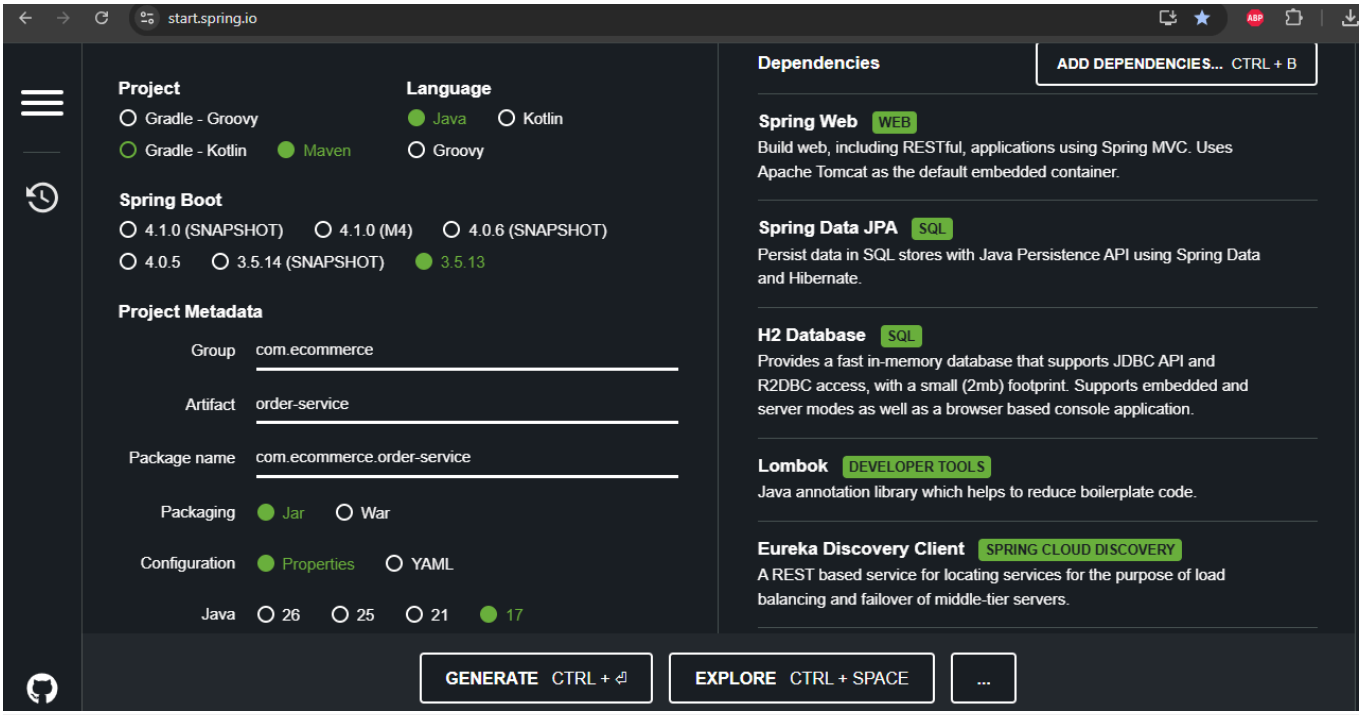
```java
package com.ecommerce.product_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

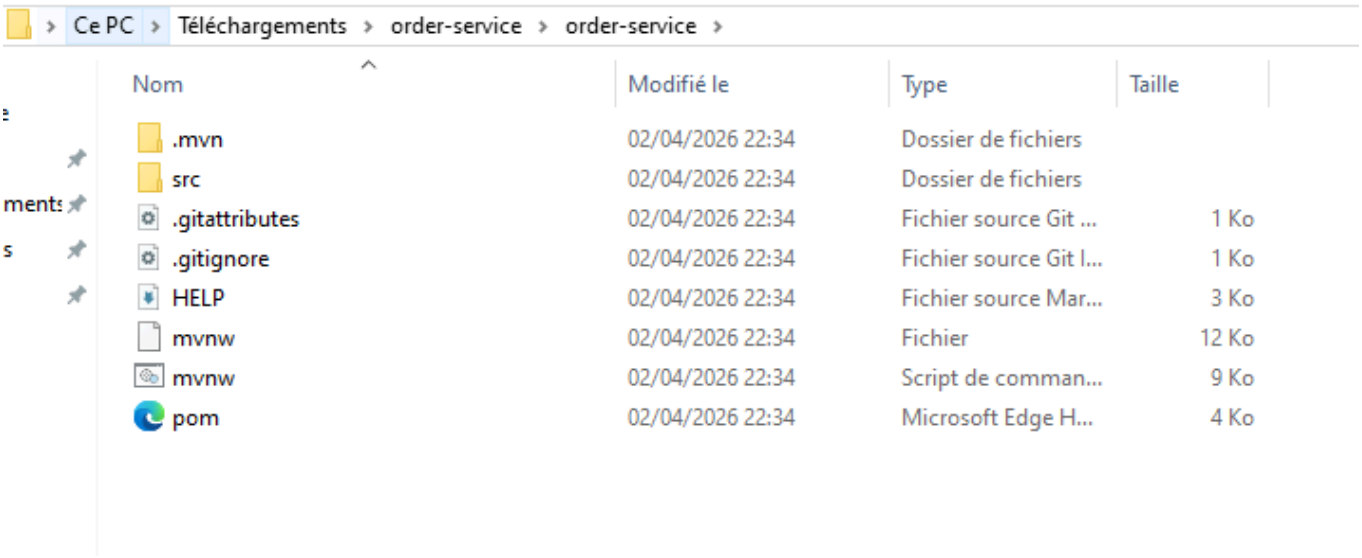
Vérification:

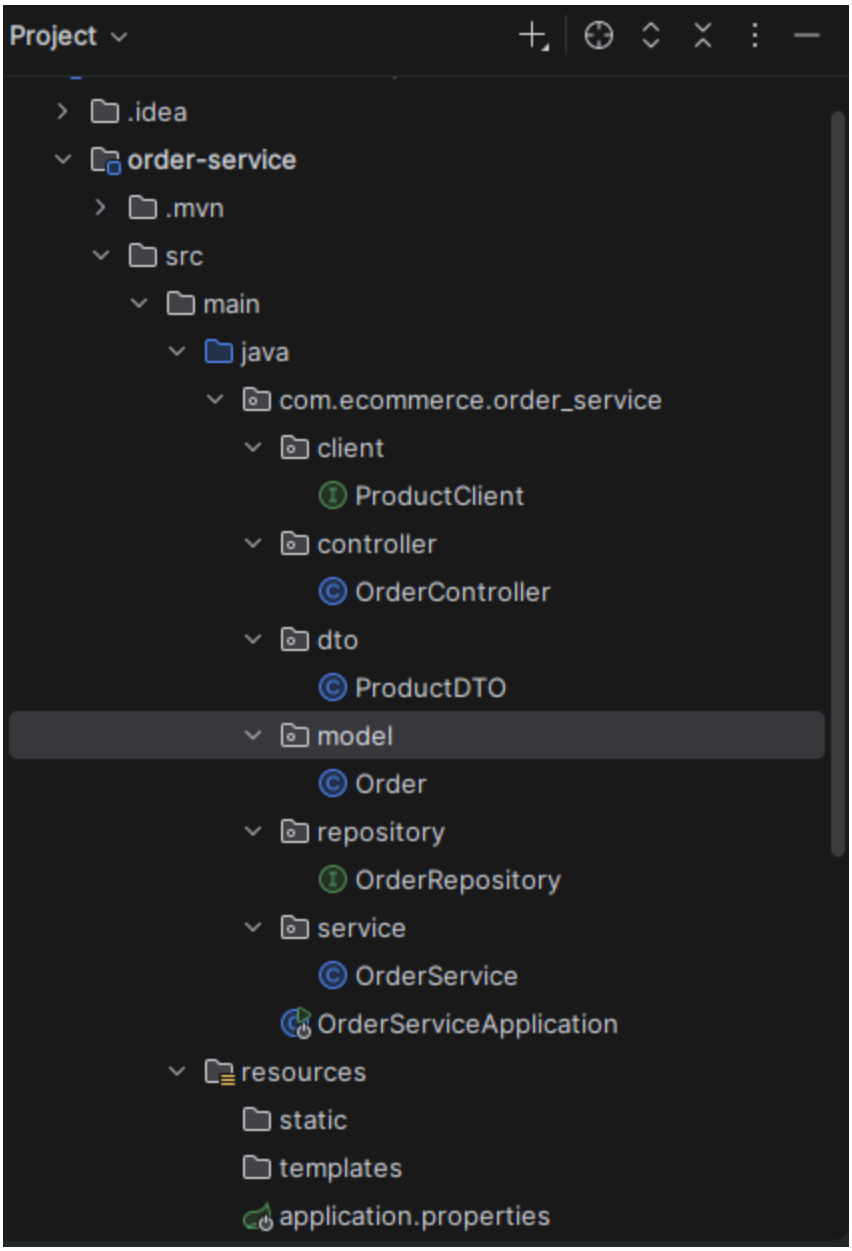
Instances currently registered with Eureka

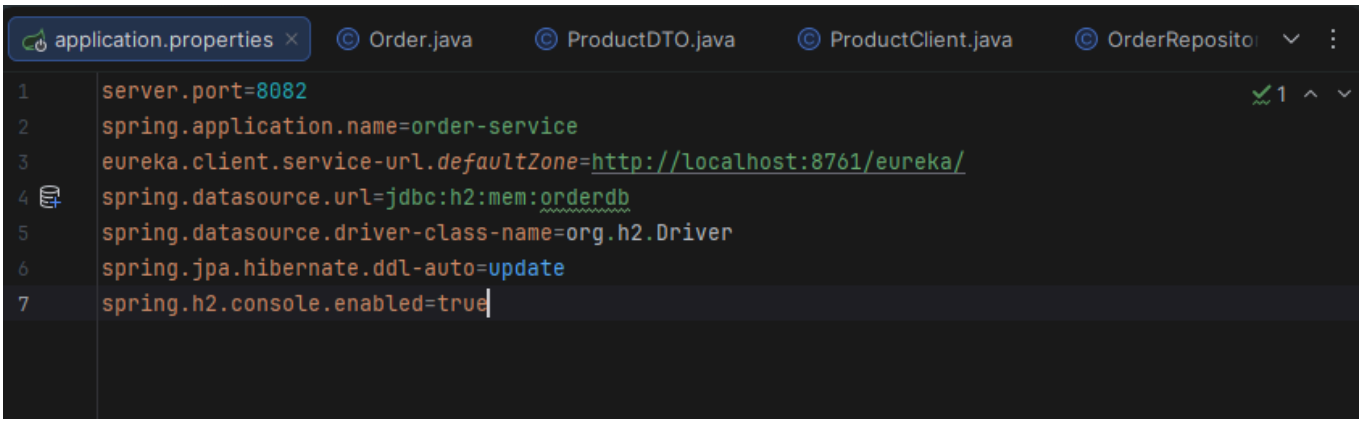
Application	AMIs	Availability Zones	Status
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-O8B7357:product-service:8081

Étape 3 : Créer le service Order:


start.spring.io
Project
Gradle - Groovy
Gradle - Kotlin
Maven
Language
Java
Kotlin
Groovy
Spring Boot
4.1.0 (SNAPSHOT)
4.1.0 (M4)
4.0.6 (SNAPSHOT)
4.0.5
3.5.14 (SNAPSHOT)
3.5.13
Project Metadata
Group
com.ecommerce
Artifact
order-service
Package name
com.ecommerce.order-service
Packaging
Jar
War
Configuration
Properties
YAML
Java
26
25
21
17
Dependencies
ADD DEPENDENCIES... CTRL + B
Spring Web
WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
Spring Data JPA
SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
H2 Database
SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.
Lombok
DEVELOPER TOOLS
Java annotation library which helps to reduce boilerplate code.
Eureka Discovery Client
SPRING CLOUD DISCOVERY
A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.
GENERATE CTRL + ⏎
EXPLORE CTRL + SPACE
...


Ce PC › Téléchargements › order-service › order-service ›
Nom
Modifié le
Type
Taille
.mvn
02/04/2026 22:34
Dossier de fichiers
src
02/04/2026 22:34
Dossier de fichiers
.gitattributes
02/04/2026 22:34
Fichier source Git ...
1 Ko
.gitignore
02/04/2026 22:34
Fichier source Git I...
1 Ko
HELP
02/04/2026 22:34
Fichier source Mar...
3 Ko
mvnw
02/04/2026 22:34
Fichier
12 Ko
mvnw
02/04/2026 22:34
Script de comman...
9 Ko
pom
02/04/2026 22:34
Microsoft Edge H...
4 Ko


Project
.idea
order-service
.mvn
src
main
java
com.ecommerce.order_service
client
ProductClient
controller
OrderController
dto
ProductDTO
model
Order
repository
OrderRepository
service
OrderService
OrderServiceApplication
resources
static
templates
application.properties


application.properties
Order.java
ProductDTO.java
ProductClient.java
OrderRepositor
server.port=8082
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true

application.properties | Order.java | ProductDTO.java | ProductClient.java | OrderRepo

```java
package com.ecommerce.order_service.model;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
    private LocalDateTime orderDate;
    private String status;
}
```

application.properties | Order.java | ProductDTO.java | ProductClient.java | OrderRepositor

```java
package com.ecommerce.order_service.dto;

import lombok.*;

@Data  no usages
@NoArgsConstructor
@AllArgsConstructor
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

application.properties × | Order.java | ProductDTO.java | ProductClient.java × | OrderRepositor

```java
package com.ecommerce.order_service.client;

import com.ecommerce.order_service.dto.ProductDTO;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "product-service")  no usages
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable Long id);
}
```

ication.properties × | Order.java | ProductDTO.java | ProductClient.java | OrderRepository.java ×

```java
package com.ecommerce.order_service.repository;

import com.ecommerce.order_service.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository  no usages
public interface OrderRepository extends JpaRepository<Order, Long> {}
```

OrderRepository.java | OrderService.java | OrderController.java | OrderServiceApplication.java

```java
package com.ecommerce.order_service.service;
import com.ecommerce.order_service.client.ProductClient;
import com.ecommerce.order_service.dto.ProductDTO;
import com.ecommerce.order_service.model.Order;
import com.ecommerce.order_service.repository.OrderRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;
@Service  2 usages
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepository;
    private final ProductClient productClient;
    public List<Order> getAllOrders() {  1 usage
        return orderRepository.findAll();
    }
    public Order createOrder(Long productId, Integer quantity) {  1 usage
        ProductDTO product = productClient.getProductById(productId);
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        return orderRepository.save(order);
```

OrderRepository.java | OrderService.java | OrderController.java | OrderServiceApplication.java

```java
package com.ecommerce.order_service.controller;

import com.ecommerce.order_service.model.Order;
import com.ecommerce.order_service.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;

    @GetMapping
    public List<Order> getAllOrders() {
        return orderService.getAllOrders();
    }

    @PostMapping
    public Order createOrder(@RequestParam Long productId,
                             @RequestParam Integer quantity) {
        return orderService.createOrder(productId, quantity);
    }
}
```

OrderRepository.java | OrderService.java | OrderController.java | OrderServiceApplication.java

```java
package com.ecommerce.order_service;

import ...

@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Étape 4 : Tester l'application:

Démarrage des services:

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-O8B7357:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-O8B7357:product-service:8081

Tests avec Postman:

http://localhost:8081/api/products

Save | Share

POST http://localhost:8081/api/products Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "name": "Laptop",
3   "description": "Ordinateur portable haute performance",
4   "price": 1200.00,
5   "stock": 10
6 }
```

Body Cookies Headers 5 Test Results 200 OK 11 ms 268 B

{} JSON Preview Visualize

```
1 {
2   "id": 3,
3   "name": "Laptop",
4   "description": "Ordinateur portable haute performance",
5   "price": 1200.0,
6   "stock": 10
7 }
```

http://localhost:8081/api/products

Save | Share

GET http://localhost:8081/api/products Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cooki

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 5 | Test Results

200 OK · 450 ms · 480 B

{} JSON | Preview | Visualize


```
[
    {
        "id": 1,
        "name": "Laptop",
        "description": "Ordinateur portable haute performance",
        "price": 1200.0,
        "stock": 10
    },
    {
        "id": 2,
        "name": "Laptop",
        "description": "Ordinateur portable haute performance",
        "price": 1200.0,
        "stock": 10
```


http://localhost:8082/api/orders

http://localhost:8081/api/products

Save | Share

POST http://localhost:8082/api/orders?productId=1&quantity=2 Send

Docs | Params • | Authorization | Headers 8 | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON

Schema | Beautify

Body | Cookies | Headers 5 | Test Results

200 OK · 438 ms · 303 B

{} JSON | Preview | Visualize


```
{
    "id": 1,
    "productId": 1,
    "productName": "Laptop",
    "quantity": 2,
    "totalPrice": 2400.0,
    "orderDate": "2026-04-02T23:17:58.6945633",
    "status": "PENDING"
}
```

http://localhost:8082/api/orders

Save | Share

GET http://localhost:8082/api/orders Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 5 | Test Results

200 OK • 19 ms • 304 B

{} JSON | Preview | Visualize


```
[
    {
        "id": 1,
        "productId": 1,
        "productName": "Laptop",
        "quantity": 2,
        "totalPrice": 2400.0,
        "orderDate": "2026-04-02T23:17:58.694563",
        "status": "PENDING"
    }
]
```


Exercice 1 — Mettre à jour le stock:

```java
    }

    public void deleteProduct(Long id) {  1 usage
        productRepository.deleteById(id);
    }
    public Product updateStock(Long id, Integer quantity) {  no usages
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Produit non trouvé"));
        if (product.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant");
        }
        product.setStock(product.getStock() - quantity);
        return productRepository.save(product);
    }
}
```

```java
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
    @PutMapping("/{id}/stock")
    public ResponseEntity<Product> updateStock(@PathVariable Long id,
                                               @RequestParam Integer quantity) {
        return ResponseEntity.ok(productService.updateStock(id, quantity));
    }
}
```

```java
package com.ecommerce.order_service.client;

import com.ecommerce.order_service.dto.ProductDTO;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "product-service")  2 usages
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable Long id);
    @PutMapping("/api/products/{id}/stock")  no usages
    void updateStock(@PathVariable Long id, @RequestParam Integer quantity);
}
```

```java
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        return orderRepository.save(order);
    }
    public Order createOrder(Long productId, Integer quantity) {  no usages
        ProductDTO product = productClient.getProductById(productId);
        if (product.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant");
        }
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        Order saved = orderRepository.save(order);
        productClient.updateStock(productId, quantity); // ← mise à jour
        return saved;
    }
}
```